UVSim Software Requirements Specification

Milestone 4 | Team A | October 23, 2026

Scope and revision policy

The GUI opens, inspects, edits, saves, and executes BasicML programs. Editor source is distinct from execution memory. Legacy console loading remains strict; GUI loading permits malformed words for repair. More than 100 nonblank entries are rejected intact. Save requires valid words but permits an empty file. Run requires valid, nonempty source. GUI error rows refer to the displayed list after blank lines are removed.

This version supersedes the Milestone 3 final SRS for submission. FR-01 through FR-25 retain their IDs. FR-26 replaces former NFR-04 (functional recovery). Former NFR-02 is internal architecture guidance in class-definitions.md, not a customer-facing requirement. Retired NFR IDs are not reused.

Functional requirements

FR-01 | File selection

The system shall allow the user to select a BasicML text file from any accessible location through a graphical file picker.

Use cases: UC-01.

FR-02 | Editor loading

The system shall place up to 100 nonblank source entries into the GUI editor in file order, beginning at address 00, including malformed BasicML words for correction.

Use cases: UC-01, UC-15.

FR-03 | READ input

The system shall request and accept a signed integer from -9999 through 9999 through the GUI when executing READ and store the value at the specified memory address.

Use cases: UC-03.

FR-04 | Input validation and retry

The system shall reject blank, non-integer, or out-of-range READ input and preserve the pending READ instruction and prior execution state while the user corrects and resubmits the value without restarting execution.

Use cases: UC-03.

FR-05 | WRITE output

The system shall display each WRITE value in the GUI output area using four magnitude digits and a minus sign for negative values.

Use cases: UC-04.

FR-06 | Addition

The system shall add the operand memory value to the accumulator when executing ADD.

Use cases: UC-07.

FR-07 | Subtraction

The system shall subtract the operand memory value from the accumulator when executing SUBTRACT.

Use cases: UC-08.

FR-08 | Multiplication

The system shall multiply the accumulator by the operand memory value when executing MULTIPLY.

Use cases: UC-10.

FR-09 | Division

The system shall divide the accumulator by a nonzero operand memory value using integer division that truncates the quotient toward zero.

Use cases: UC-09.

FR-10 | Unconditional branch

The system shall set the instruction counter to the operand address when executing BRANCH.

Use cases: UC-11.

FR-11 | Negative branch

The system shall branch to the operand address on BRANCHNEG only when the accumulator is negative; otherwise execution shall continue sequentially.

Use cases: UC-12.

FR-12 | Zero branch

The system shall branch to the operand address on BRANCHZERO only when the accumulator equals zero; otherwise execution shall continue sequentially.

Use cases: UC-13.

FR-13 | Halt

The system shall stop executing instructions when it encounters HALT.

Use cases: UC-14.

FR-14 | Arithmetic overflow

The system shall retain only the lowest four magnitude digits of an overflowing arithmetic result, preserving its sign, as the accumulator value used by subsequent instructions.

Use cases: UC-07, UC-08, UC-09, UC-10.

FR-15 | File-error recovery

The system shall report unreadable files, decoding errors, and files exceeding 100 nonblank entries without replacing current editor contents or path.

Use cases: UC-01.

FR-16 | Instruction validation

The system shall detect instructions containing invalid operation codes.

Use cases: UC-14.

FR-17 | Execute editor contents

The system shall validate and execute current editor contents on Run, including unsaved edits and the active cell edit.

Use cases: UC-14.

FR-18 | Execution status

The GUI shall indicate when program execution has completed.

Use cases: UC-14.

FR-19 | Error display

The GUI shall display an error message when an error occurs.

Use cases: UC-02, UC-14.

FR-20 | Word validation

The system shall require each entry to contain a sign and four decimal digits in -9999 through +9999 before execution or saving; identify the invalid editor row/address and prevent the operation if validation fails.

Use cases: UC-02.

FR-21 | Fresh execution

The system shall reset execution memory, registers, halt state, pending READ input, and output from current validated editor words before each successful Run.

Use cases: UC-14.

FR-22 | Stop execution

The system shall allow the user to stop a BasicML program while it is executing.

Use cases: UC-26.

FR-23 | LOAD

The system shall copy the value from the specified memory address into the accumulator when executing LOAD.

Use cases: UC-05.

FR-24 | STORE

The system shall copy the accumulator value into the specified memory address, replacing its previous contents, when executing STORE.

Use cases: UC-06.

FR-25 | Output retention

The system shall retain output after HALT, Stop, runtime errors, and opening another source, clearing it only when a valid fresh Run begins.

Use cases: UC-04, UC-14.

FR-26 | Repeat execution and recovery

The system shall allow a fresh Run after HALT, Stop, or an error without reopening the file or restarting the application.

Use cases: UC-14, UC-26.

FR-27 | View entries

The system shall display instructions and data in an editable list with consecutive addresses 00 through 99.

Use cases: UC-15.

FR-28 | Add entries

The system shall insert +0000 before the first selected row, or append if none is selected, and renumber subsequent addresses.

Use cases: UC-16.

FR-29 | Modify entries

The system shall allow individual source entries to be modified and retain malformed manual edits for correction with an error indication.

Use cases: UC-17.

FR-30 | Delete entries

The system shall delete selected rows and renumber remaining entries consecutively.

Use cases: UC-18.

FR-31 | Clipboard operations

The system shall support Cut, Copy, and Paste of one or multiple entries through editor controls and keyboard shortcuts; reject invalid clipboard words without modifying the editor.

Use cases: UC-19.

FR-32 | Capacity limit

The system shall reject Add, Paste, or file Open operations that exceed 100 entries without truncating or partially replacing current contents.

Use cases: UC-20.

FR-33 | Execution interlock

The system shall disable editor mutations, Open, Save, Save As, and Run while running or awaiting READ; Copy and Stop shall remain available.

Use cases: UC-21.

FR-34 | Save current program

The system shall save validated editor words to the current path, one formatted word per line, and mark source saved only after success; with no path, Save shall request a destination.

Use cases: UC-22.

FR-35 | Save As

The system shall allow a user-selected filename and accessible directory through Save As, updating the current path only after a successful write.

Use cases: UC-23.

FR-36 | Save failure recovery

The system shall preserve source, dirty state, and current path when saving fails or its dialog is canceled; failed replacement shall not leave a partial file in place of the original.

Use cases: UC-22, UC-23.

FR-37 | Unsaved-change decisions

The system shall offer Save, Discard, and Cancel before Open or Close discards edited source; a canceled or failed requested save shall prevent the pending Open or Close.

Use cases: UC-25.

FR-38 | Default branding

The system shall use primary #4C721D and off-color #FFFFFF when no valid custom theme is available.

Use cases: UC-24.

FR-39 | Configurable branding

The system shall read primary and off_color #RRGGBB settings from the application theme.ini at startup and apply both colors without recompilation or reinstallation.

Use cases: UC-24.

FR-40 | Theme fallback

The system shall restore the whole UVU palette when configuration is missing, unreadable, malformed, incomplete, or contains an invalid color.

Use cases: UC-24.

FR-41 | Source preservation

The system shall execute using machine memory separately from editor source, preserving editor contents and saved/unsaved state.

Use cases: UC-14.

FR-42 | Blank and empty sources

The system shall ignore blank lines and surrounding whitespace during GUI loading; permit editing/saving empty source but reject it on Run. GUI validation line numbers shall refer to displayed entries after blank lines are removed.

Use cases: UC-01, UC-02, UC-22.

Non-functional requirements

NFR-01 | Responsiveness

The system shall remain responsive to GUI controls, including Stop, during program execution and yield to the GUI event loop after no more than 100 instructions in a single scheduled execution batch.

NFR-03 | Dependencies

The system shall run using Python 3.10 or newer with its standard library and Tcl/Tk support, without requiring third-party Python packages.

NFR-05 | Execution response time

The system shall begin execution within 2 seconds after Run is selected for an already loaded instructor test program on the supported desktop test environment.

NFR-06 | Usability

The system shall provide clearly labeled GUI controls, prompts, input fields, and output areas so users can understand their purpose.

NFR-07 | Default-theme readability

Default UVU colors shall keep labels, input, and output readable in the supported desktop GUI. Custom-color contrast is the user's responsibility.

Verification

The accompanying test register maps every current automated method to use cases. The recorded run has 112 passing methods. Passing methods demonstrate their assertions, not every possible input or platform. NFR-05 retains the earlier two-second target; a dedicated timing acceptance measurement is not recorded. Final manual readability/visual acceptance and fresh-clone verification remain listed in verification.md.